

Resultados — Ejercicio 02: Ensamblas, reducción y Green AI

INF-8239 · Ciencia de Datos II · Repositorio: github.com/DAMB18/SVM-con-pipeline (tag **u01-ejercicio02**, commit **c587ff3c**)

Identificación del dataset

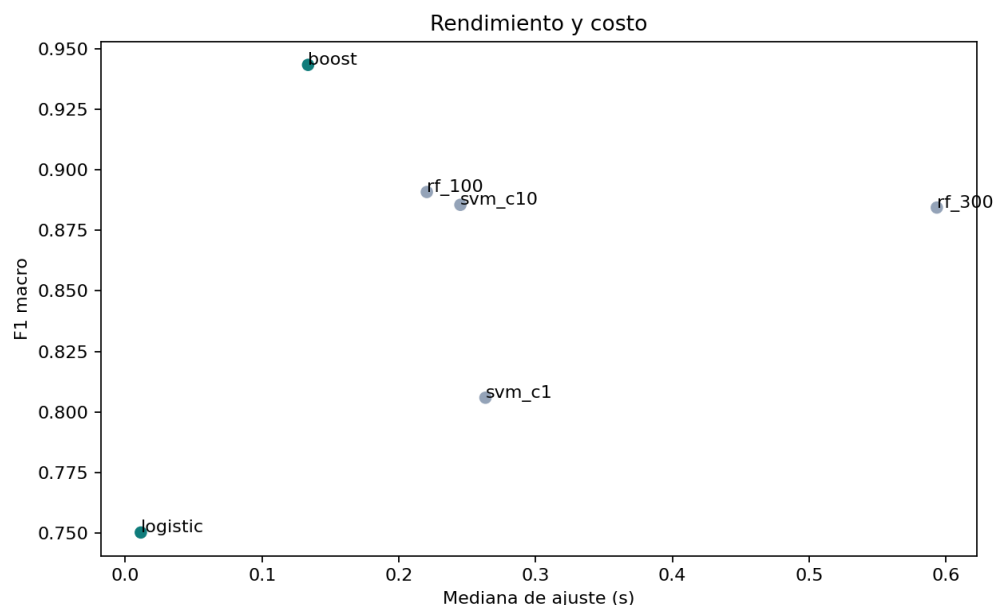
Iranian Churn Dataset (UCI Machine Learning Repository) — mismo dataset, target y partición aprobados en el Ejercicio 01. Ficha: archive.ics.uci.edu/dataset/563/iranian+churn+dataset. Descarga (ZIP público, sin autenticación): archive.ics.uci.edu/static/public/563/iranian+churn+dataset.zip. Licencia CC BY 4.0. 3,150 clientes, 14 columnas tras retirar *Age* (redundante con *Age Group*). Target **Churn** (binario, ~15.7% clase positiva). Partición 2,520/630, estratificada, **random_state=42** — idéntica a LAB02.

Tabla principal de resultados

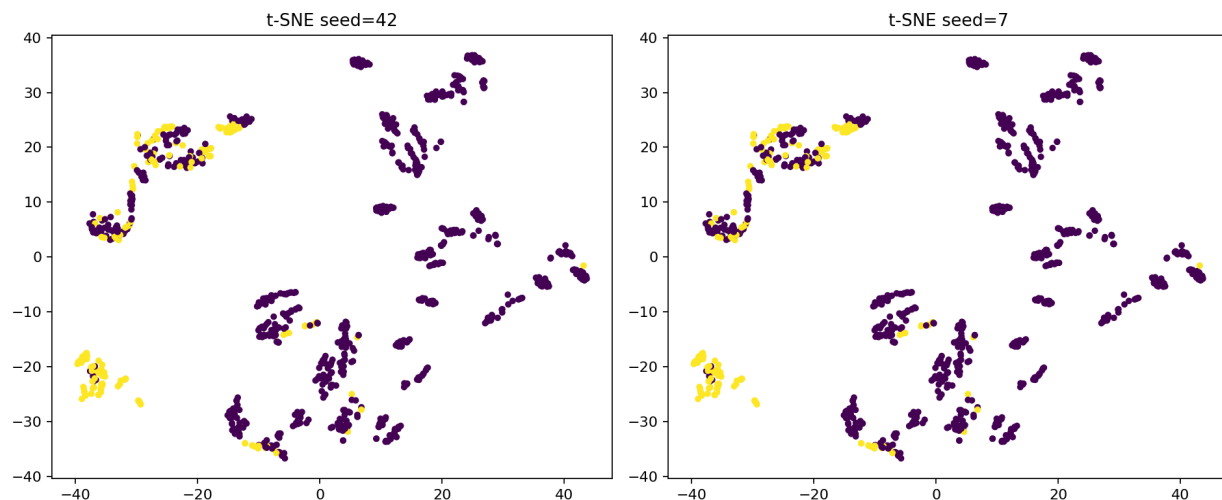
Modelo	Familia	F1 macro	Recall Churn=1	Ajuste (s)	Inferencia (ms)	Tamaño (KB)	Pareto
boost	HistGradBoosting	0.943	0.909	0.144	5.6	316.7	sí
rf_100	Random Forest (100)	0.891	0.747	0.216	35.8	995.3	no
svm_c10	SVM C=10	0.886	0.697	0.244	12.5	62.5	no
rf_300	Random Forest (300)	0.884	0.737	0.600	77.3	2971.9	no
svm_c1	SVM C=1	0.806	0.495	0.262	15.3	78.0	no
logistic	Reg. logística	0.750	0.424	0.012	2.4	4.1	sí

PCA (95% de varianza): retiene 8 de 14 columnas; F1 macro del SVM baja de 0.806 a 0.799 (−0.007). Fuente completa: [reports/green_ai_results.csv](#).

Figuras



Frontera de Pareto (F1 macro vs. mediana de tiempo de ajuste). Solo **boost** y **logistic** quedan sin dominar; los otros cuatro modelos son estrictamente peores que boost en ambas dimensiones a la vez.



t-SNE con dos semillas (42 y 7) sobre 1,000 clientes. La estructura gruesa se mantiene entre semillas; cambia la orientación y algunos detalles finos del embedding. No prueba causalidad ni sustituye la evaluación real.

Conclusión — defensa de la decisión Pareto

Con las seis configuraciones evaluadas, la frontera de Pareto entre F1 macro y mediana de tiempo de ajuste queda formada por solo dos modelos: **boost** (HistGradientBoosting) y **logistic** (regresión logística). Los otros cuatro —rf_100, svm_c10, rf_300 y svm_c1— quedan estrictamente dominados por boost en ambas dimensiones a la vez, así que no tiene sentido elegirlos bajo ningún balance razonable entre desempeño y costo.

El F1 macro máximo lo obtiene boost, con 0.943, casi diez puntos por encima del segundo mejor no dominado por él (rf_100, 0.891), y lo logra con una mediana de ajuste de apenas 0.144 s — más rápida que las dos SVM y que ambos Random Forest.

La alternativa más barata de la frontera es logistic: 0.012 s de ajuste (91.7% menos tiempo que boost) y el modelo más liviano (4.1 KB contra 316.7 KB). Pero detecta solo el 42% de los clientes que realmente se van (recall_churn=0.424), contra 91% con boost (0.909) — una diferencia absoluta de F1 macro de 0.193 puntos. En términos del error más costoso del proyecto (el falso negativo de retención), logistic deja sin detectar más de la mitad de los clientes que se van, contra menos de una décima parte con boost.

Decisión: me quedo con boost, no con la opción más barata de la frontera. 0.144 s y 316 KB son costos triviales en términos absolutos para un proceso de scoring diario sobre unos pocos miles de clientes; el ahorro de 91.7% de tiempo que ofrece logistic no compensa perder más de la mitad del recall sobre la clase que importa.

Limitaciones: tiempo y tamaño son proxies de costo computacional, no consumo energético ni huella de CO2 —no se midió ninguna de las dos—. Los tiempos son relativos a esta máquina y a esta corrida (Linux x86_64, scikit-learn 1.8.0, sin otros procesos compitiendo por CPU); en otro hardware la magnitud cambiaría aunque el orden relativo entre modelos probablemente se mantenga.

Pruebas: tests/test_green.py (4 casos de pareto_flags) + los 8 de Ejercicio 01 = 12 pasando. README.md actualizado con la sección LAB03.